



Version control using Git & GitHub

Pablo Orviz Fernández <orviz@ifca.es>

Introducción a la Ciencia de Datos Masivos y a la Ciencia en Abierto - Máster en Ciencia de Datos



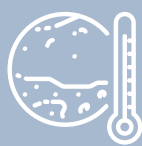
COURSE CONTENTS



01 **VERSION CONTROL**



02 COLLABORATION PLATFORM: GITHUB



03 HANDS-ON GIT





VERSION CONTROL

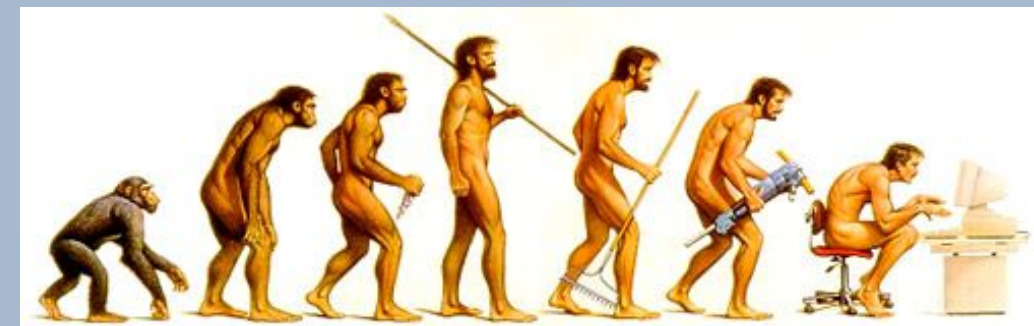
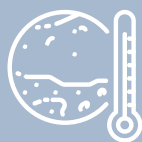


"Systems that record changes to a/set of file/s over time so that you can recall specific versions later" (Chacon&Straub, Pro Git)



5 key reasons to use version control

1. **Document Evolution:** Keep a historical, chronological record of your software's growth.
2. **Time Travel:** Revert to previous code states and revert or recover deleted files.
3. **Chaos-Free Collaboration:** Work on the same file with multiple people simultaneously.
4. **Isolate Experiments:** Test new features in separate branches without affecting the main project.
5. **Project Auditing:** You want to check when & who & last modified something that is causing a problem/issue.





VERSION CONTROL



"Systems that record changes to a/set of file/s over time so that you can recall specific versions later" (Chacon&Straub, Pro Git)

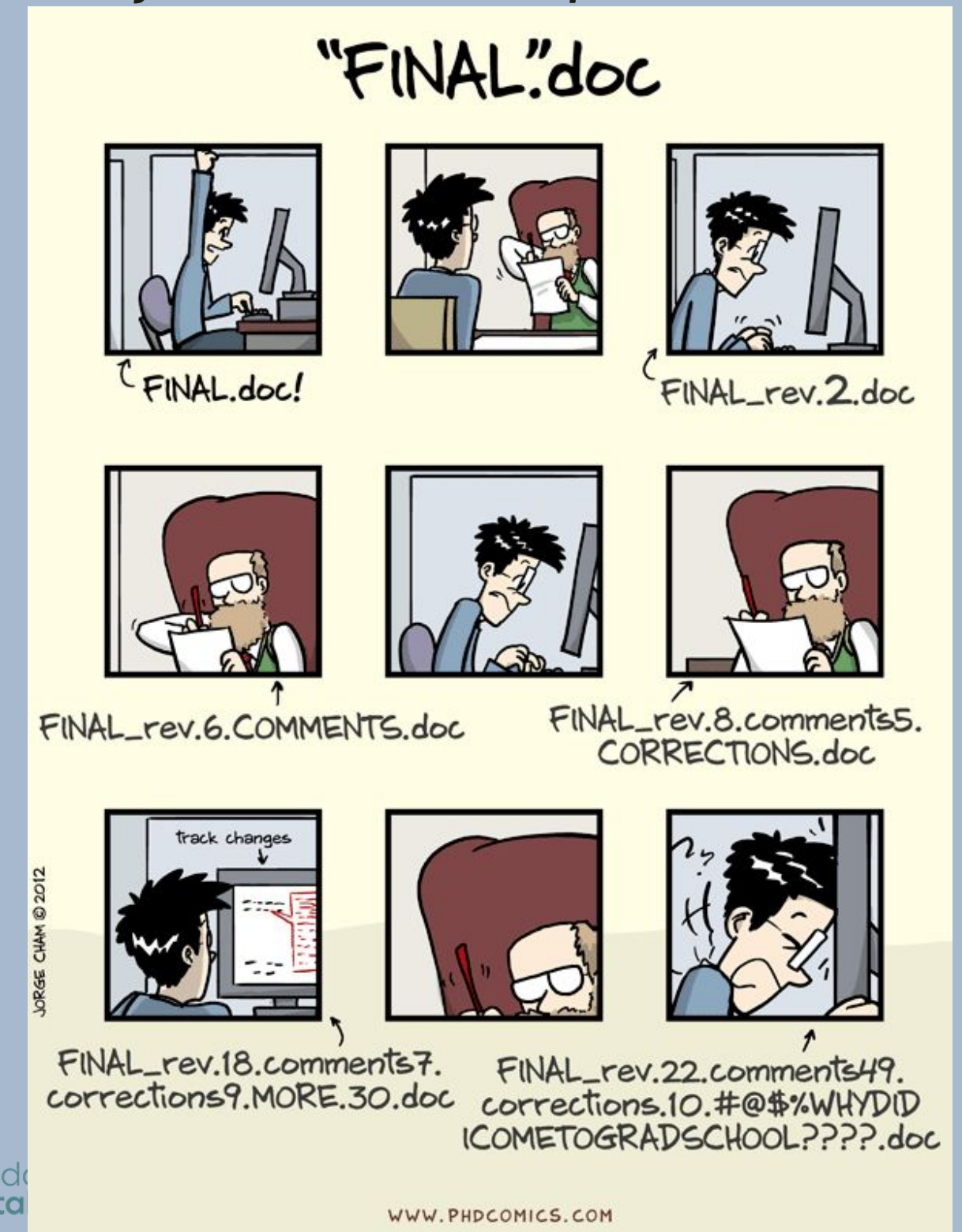
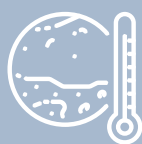


Myth vs. Reality

The Myth: "I can manage versions just fine by renaming my files (e.g., script_v2_final_fixed_DEFINITIVE.py)."

The Reality: Manual renaming leads to **accidental overwrites**, and makes it **impossible to compare changes** or know what actually changed between "final" and "final_v2".

The Value of Version Control (Git): Git tracks the content inside the files, not the file names, and **remembers every single snapshot of its history** behind the scenes.





VERSION CONTROL



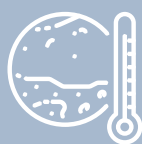
"Systems that record changes to a/set of file/s over time so that you can recall specific versions later" (Chacon&Straub, Pro Git)



Myth vs. Reality



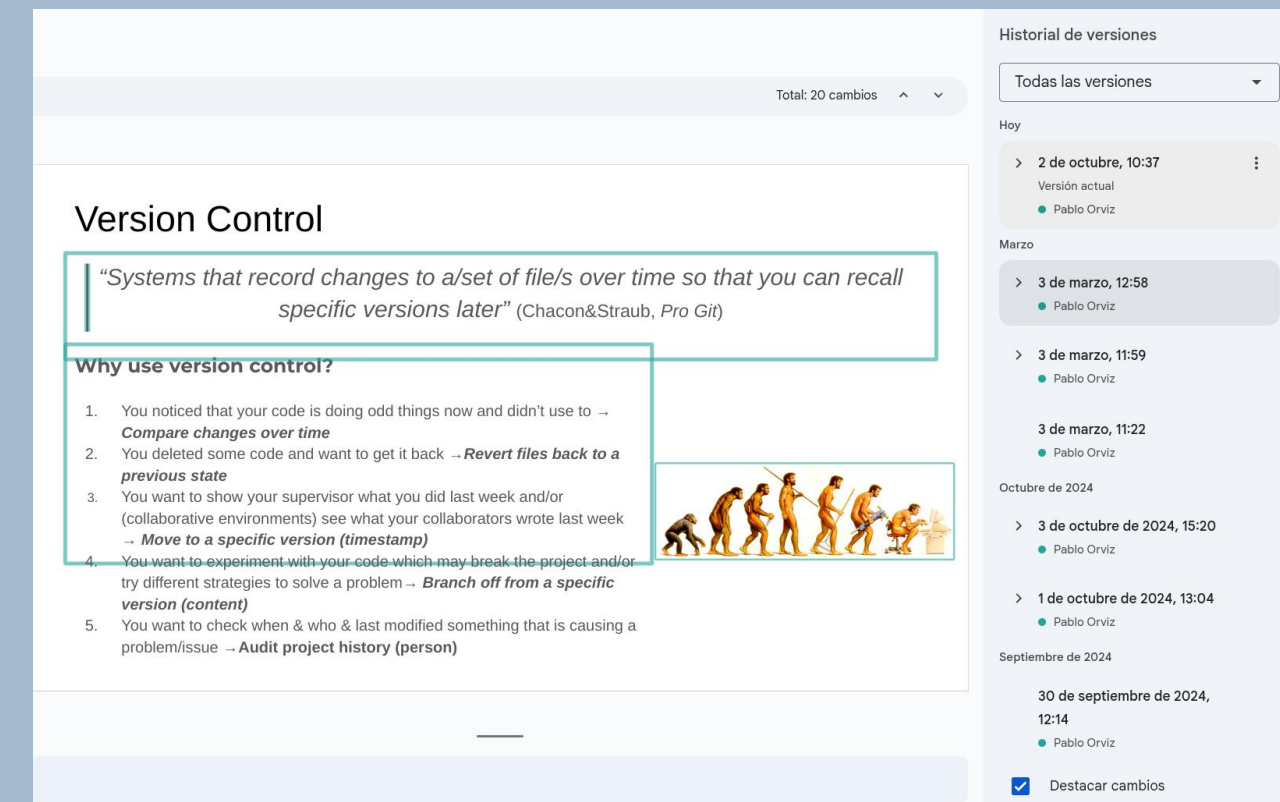
The Myth: "Google Docs or OneDrive already save my versions automatically."



The Reality: Word processors don't understand code. They **blindly merge changes and delete old history** to save space.



The Value of Version Control (Git): A professional tool that guarantees **absolute traceability, intentional manual commits, and the ability to safely collaborate** without overwriting each other's work.



"The revisions for your file may occasionally be merged to save storage space" (Google Docs disclaimer)

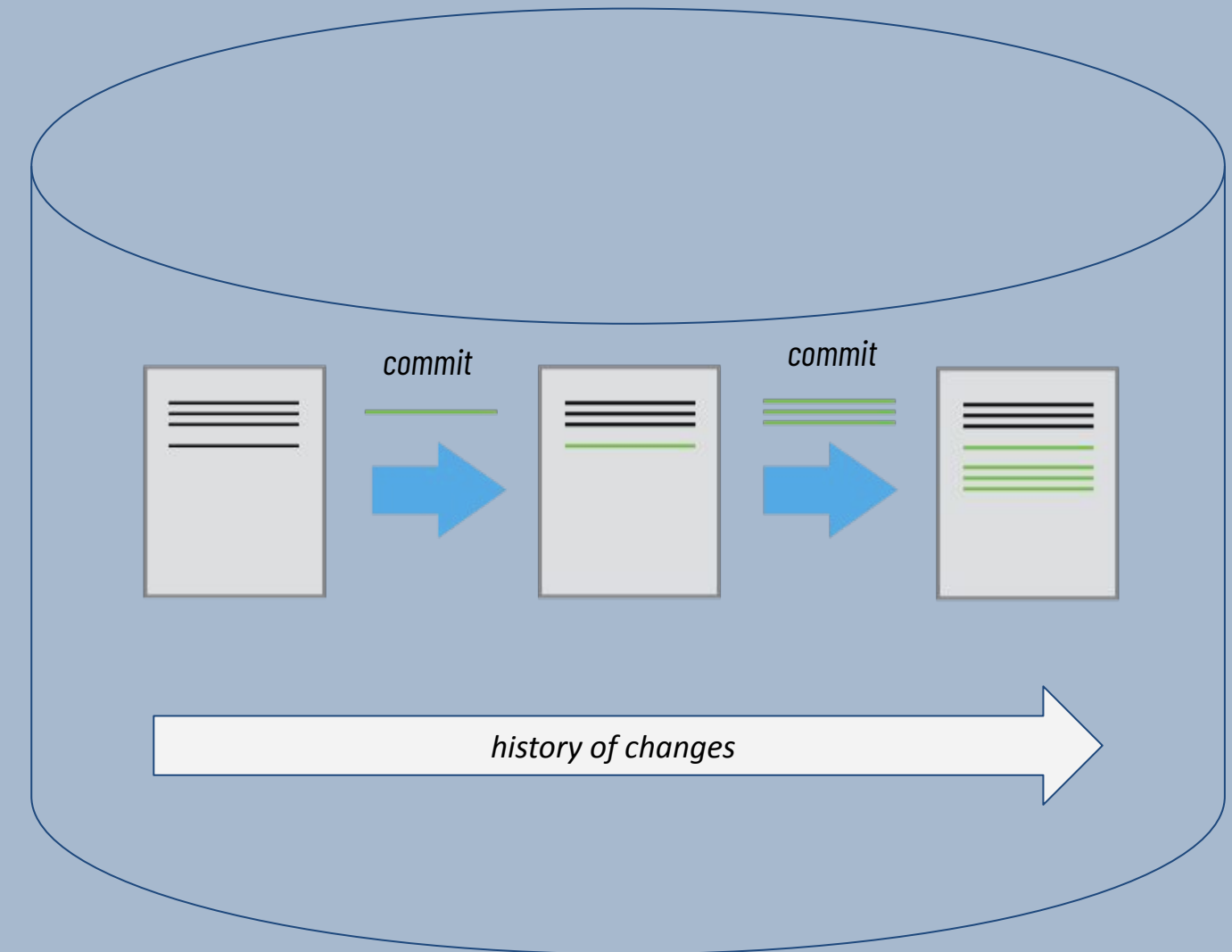
HOW VERSION CONTROL WORKS

Core concepts

The **Repository (Repo, Project)**: the main database/folder where the VC System (VCS) tracks every single file/directory.

The **History**: single-source-of-truth that maintains a linear, chronological timeline with the evolution of the changes in the repository.

The **Commit**: snapshot of the content of your files/directories at a specific moment in time.



HOW VERSION CONTROL WORKS

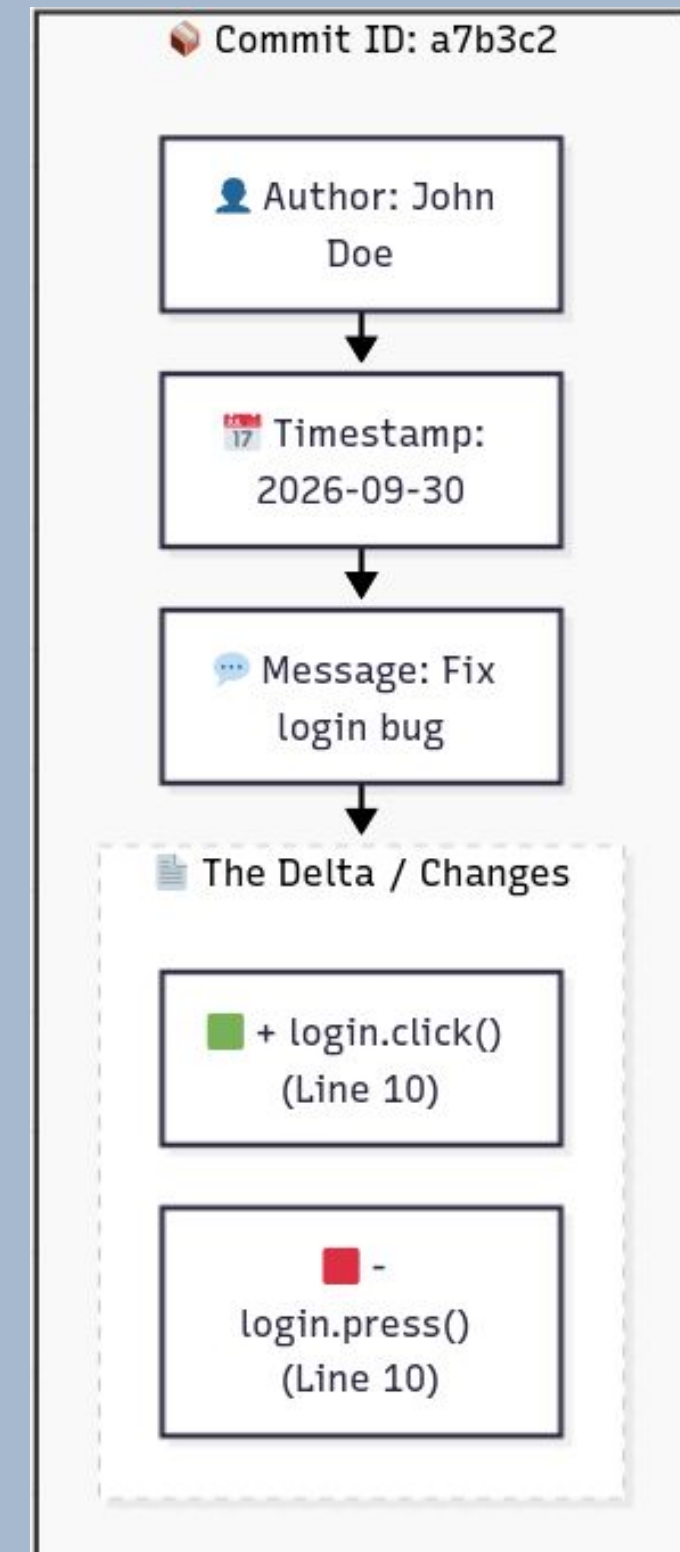
Every **Commit** is bundled with crucial information (**metadata**)

- Who made the change (Author)
- When it was made (Timestamp)
- Why it was made (Commit Message)
- ...



Core concept

The **Delta**: Instead of duplicating the whole folder, the system **only records what was added (+) or removed (-)**.



HOW VERSION CONTROL WORKS

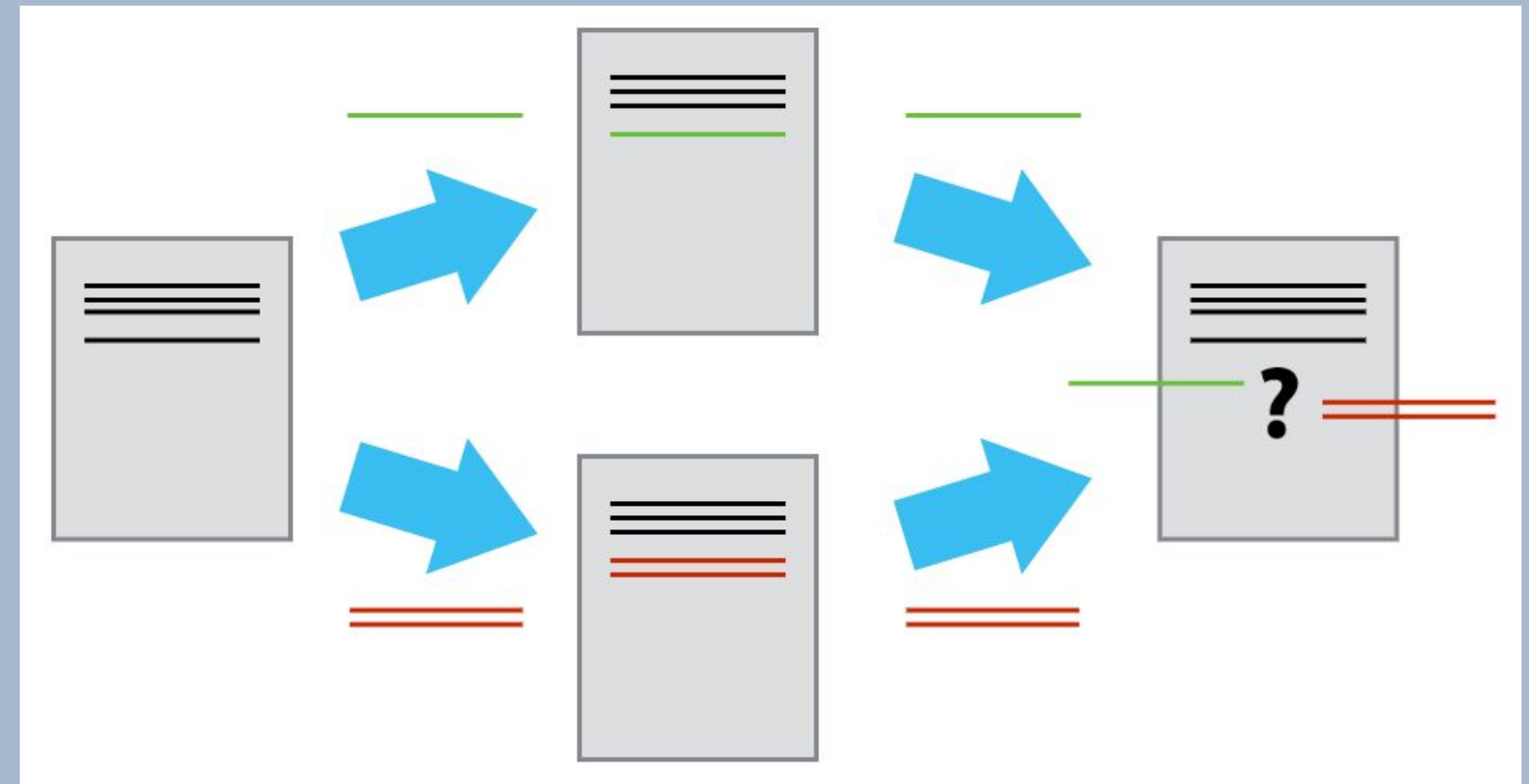
Core concept: the Merge Conflict

The Scenario: Two developers **modify the exact same line of the exact same file**.

The Dilemma: When trying to combine both versions, the system cannot guess **which change is the correct one**.

The Halt: The system stops the merge, flags the file, and prevents any accidental overwrites.

The Resolution: A **human must review the code, choose the final version**, and manually clear the conflict.



HOW VERSION CONTROL WORKS

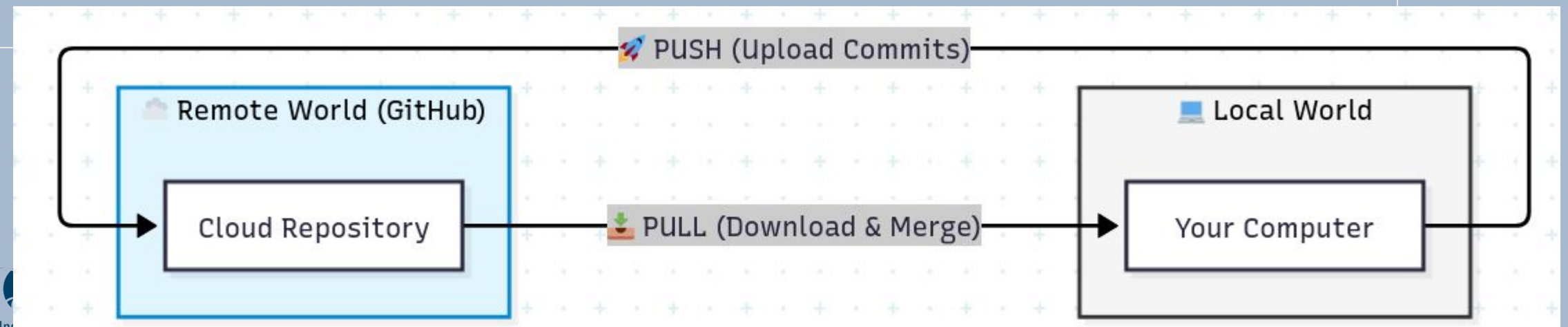
The **Two Worlds**: Your project lives in two places

- **Local Repository**: On your own computer (offline, private, fast).
- **Remote Repository**: On a cloud server like GitHub (online, shared, backup).

Core concepts

The **Push action** (Uploading): Sending your local commits to the remote repository so your team can see them.

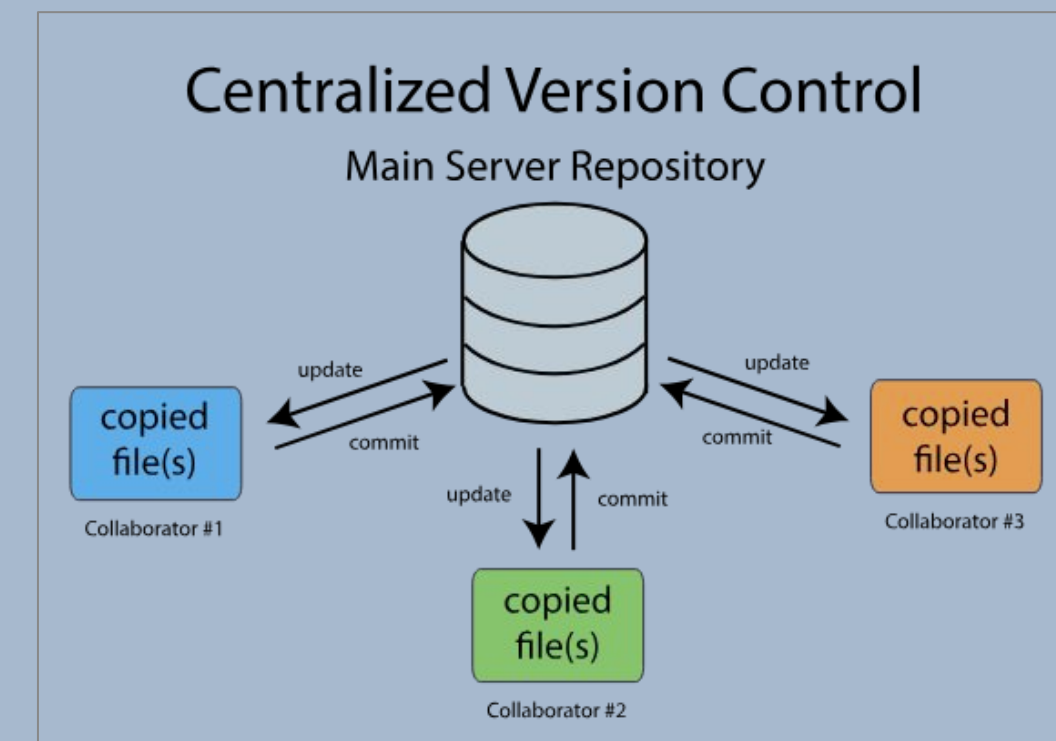
The **Pull action** (Downloading, Checkout): Fetching the latest changes from the remote repository and merging them into your computer.



(BRIEF) HISTORY OF VERSION CONTROL SYSTEMS

The Centralized VCSs (The Server-Client Era)

- **Context:** Born in the early 1980s (e.g., CVS, Subversion/SVN) to **allow collaboration among geographically distributed teams**.
- **Architecture:** A **single central server** holds all versioned files. **Clients "check out" only the specific files they need** to work on.
- **Lock Mechanism:** meaning often **only one developer could edit a file at a time**.
- **Pros:** Huge step ahead from manual backups; **clear visibility** of who is doing what.
- The Fatal Flaw (**Cons**):
 - **Single Point of Failure**
 - If the server goes down, no one can work.
 - If the disk corrupts, the entire project history is lost forever.
 - **Slow Performance:** Every operation requires a network connection to the remote server.



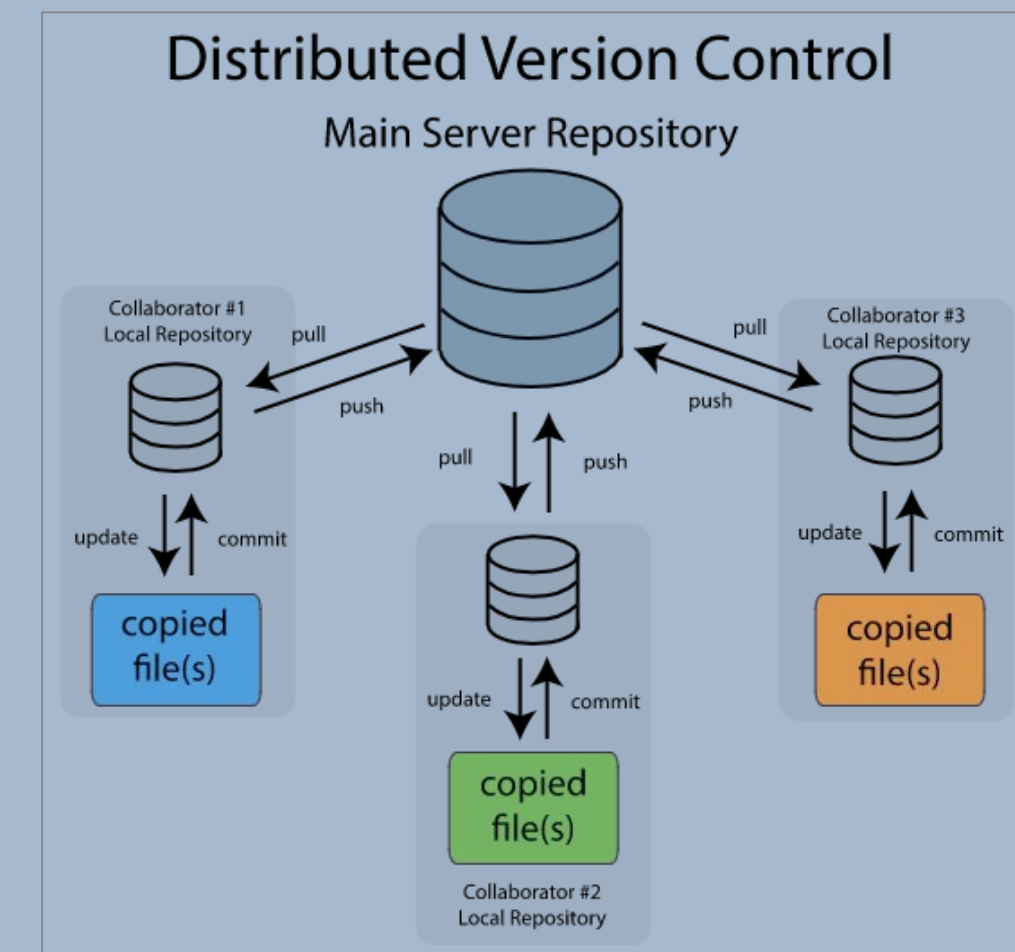
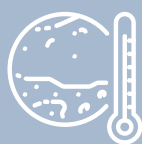


(BRIEF) HISTORY OF VERSION CONTROL SYSTEMS



The Distributed VCSs (Modern Collaboration)

- The **Paradigm Shift**: Modern systems (e.g., Git, Mercurial, Fossil) **remove the dependency on a single central server**.
- The **Architecture**: Every single client clones the entire repository, including the full historical timeline. Every **repository is a complete backup**.
- **Concurrent Work: No more file locking**. Thanks to powerful merging tools, multiple authors can work on the exact same files simultaneously.
- **Pros**:
 - **High Availability**: If a server fails, the project can be **fully restored from any developer's repository**.
 - **Blazing Fast**: All **commits, history logs, and branches are processed locally** on your computer (no internet needed).
- **Cons**: None! (Almost... it requires a slightly steeper learning curve).



(BRIEF) HISTORY OF VERSION CONTROL SYSTEMS

The **Birth of Git** (Context & Lore)

- **The Timeline:** By the early 2000s, older tools (RCS, CVS) became legacy systems due to limited capabilities.
- **The Catalyst (2005):** Linus Torvalds created Git in just a few weeks as an open-source alternative to BitKeeper to manage the massive development of the Linux Kernel.
- **What does "Git" mean?:** Torvalds famously provided multiple definitions in the [project's README](#) (depending on his mood): "Global Information Tracker", "random three-letter combination that is pronounceable", "goddamn idiotic truckload of sh*t"
- **Current status:** Today, Git is the undisputed global standard for software development.



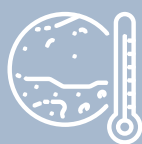


(BRIEF) HISTORY OF VERSION CONTROL SYSTEMS



Alternatives to Git: **Fossil**

- **Designed for Different Scales:** Git was made for the chaotic Linux Kernel (thousands of developers); **Fossil for SQLite (smaller, high-trust team)**
- **Software Scope:** Git tracks code only (GitHub complements it). **Fossil is "GitHub-in-a-box"** (includes built-in wiki, tickets, and forum).
- **Storage & Data:** Git uses a custom key/value store. **Fossil uses a queryable SQLite database.**
- **History of Versions' Philosophy:** Git allows rewriting history (rebase/squash) to look perfect. **Fossil records history exactly as it happened.**
- **Workflow:** Git relies on manual pushes. **Fossil uses automatic synchronization (autosync) by default**

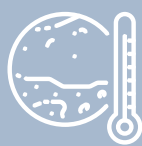




VCS IN A NUTSHELL (SUMMARY)



- **The Core Need:** Manual file renaming (e.g., `_final_v2`) creates chaos. Version Control Systems (VCS) provide absolute tracking, history, and safety for your code.
- **Key Mechanisms:**
 - **Commits:** Intentional "snapshots" of your project, not silent background autosaves.
 - **Deltas:** The system **only records the difference**: lines added (+) or removed (-), keeping it lightweight.
- **The Evolution:** from Centralized systems (high risk of server failure, slow remote operations) to **modern Distributed systems** (every machine holds a full backup, ultra-fast local operations).
- **Collaboration** relies on **moving changes between your Local Repository (your computer) and the Remote Repository** (the cloud, e.g. GitHub) using Push and Pull.
- **Different Philosophies:**
 - **Git** (follows: classic Unix design principle: "Do one thing and do it well."): The de-facto tool for control version based on its **powerful, modular code-tracker designed for massive, chaotic projects** that relies on external platforms like GitHub,
 - **Fossil** (follows: Cathedral Philosophy) An all-in-one "GitHub-in-a-box" database designed for small teams, prioritizing an unchangeable history.





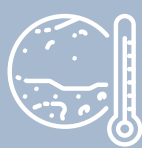
COURSE CONTENTS



01 VERSION CONTROL



02 COLLABORATION PLATFORM: GITHUB



03 HANDS-ON GIT





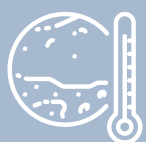
THE RISE OF SOCIAL CODING



- **The Shift in Collaboration:** Modern platforms (GitHub, GitLab) transformed code hosting into a **global social network for developers**.
- **The Core Impact:** They **shifted the focus from central organizations to individual developers**, democratizing peer production.



Before Social Coding	With Social Coding
Rigid Hierarchy: Strict separation between code producers and consumers.	Fluid Framework: Consumers are producers. Anyone can contribute.
High Barrier: Only professional developers or core maintainers could easily contribute.	Low Barrier: Forking and Pull Requests made contributing accessible to everyone.
Centralized Control: Heavy gatekeeping on what gets built and merged.	Decentralized Innovation: The community drives the evolution of the software.



GitHub didn't just host code; it became the world's resume for developers and the town square for open source.





WHAT GITHUB ADDS TO GIT



Collaboration & Social Features:

- **Pull Requests (PRs):** The ultimate tool for code review, discussion, and testing before merging code.
- **Forks:** One-click cloning of any public repository into your own account to start experimenting or contributing.



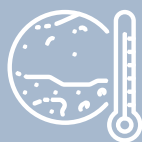
Documentation & Hosting:

- **GitHub Pages:** Turn any repository into a public website directly from your code
- **Wikis:** A built-in, separate section for long-form documentation, guides, and developer manuals.



Access & Collaboration Models:

- **Public vs. Private Repositories:** Public repos are open to the world for open-source collaboration.



Project Management Tools:

- **Issues & Discussions:** Integrated bug tracking, feature requests, and forum-style team communication.
- **GitHub Projects:** Kanban-style boards (like Trello) to plan milestones and track team progress.



Automation & Operations:

- **GitHub Actions (CI/CD):** Automatically test, build, and deploy your code every time you make a change.
- **Security & Insights:** Automatic alerts for vulnerable dependencies and detailed repository analytics.



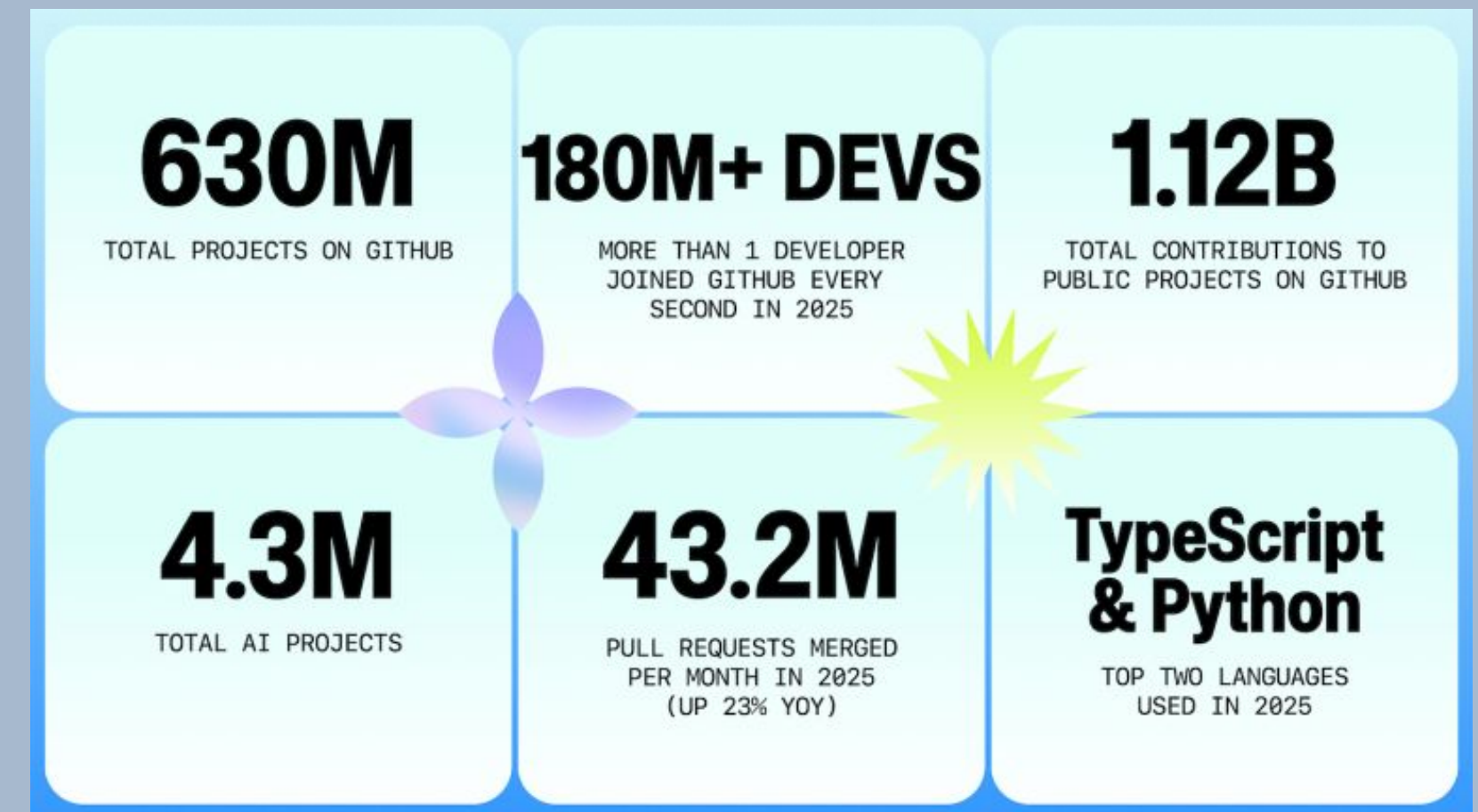
GITHUB IN NUMBERS

The Developer Tsunami: GitHub now hosts over 180 million developers globally.

Steady Growth: More than 36 million new developers joined in a single year – which averages to ~1 new developer signing up every single second.

De-facto Platform for Code Hosting: The platform hosts 630 million total repositories, with 121 million new ones created just last year.

Open Source Dominance: 63% of all repositories on the platform are public or open-source.



Further details in the [Octoverse 2025 report](#).

SUCCESS SCIENCE STORIES IN GITHUB

COVID-19 Genomics & Tracking

The Challenge: Mapping the global spread of a pandemic in absolute real-time.

GitHub impact: the Johns Hopkins University used a public GitHub repository to host the daily data. It **became the world's single source of truth** for governments, researchers, and media worldwide.

[https://doi.org/10.1016/S1473-3099\(20\)30120-1](https://doi.org/10.1016/S1473-3099(20)30120-1)

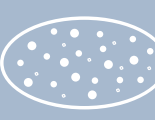
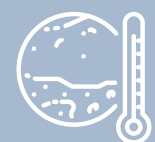


The Event Horizon Telescope (First Black Hole Image)

The Challenge: Processing petabytes of scattered data from telescopes all over the world.

GitHub impact: Hundreds of international scientists collaborated on the NumPy and SciPy Python libraries hosted on GitHub to build the imaging algorithms.

<https://doi.org/10.3847/2041-8213/ab0e85>



GITLAB AS AN ALTERNATIVE TO GITHUB

Feature	GitHub 	GitLab 
Primary Focus	Social coding , community size, and open-source hosting [2.1].	Enterprise DevOps , CI/CD automation, and project governance.
CI/CD Integration	GitHub Actions (powerful, modular, community-driven).	Built-in CI/CD (industry pioneer, native, highly stable).
Extensibility	Third-party Apps: You pick extensions from a vast Marketplace.	All-in-One: Most features are native and built into the platform.
Self-Hosting	Available (GitHub Enterprise), but complex and cloud-oriented.	Excellent self-hosting: Easy to deploy on private company servers.
Open Source Model	Proprietary platform owned by Microsoft.	Open-Core (community edition code is public).





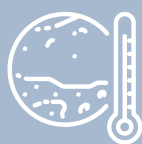
COURSE CONTENTS



01 **VERSION CONTROL**



02 **COLLABORATION PLATFORM: GITHUB**



03 **HANDS-ON GIT**





GETTING STARTED WITH GIT

Our approach in this course: **Command-line (CLI) over Graphical Interfaces (GUI)**

The GUI limitations:

1. **GUIs implement only a subset of Git features** (e.g. GitHub Desktop).
2. With GUIs you **won't understand the real Git workflow**.
3. GUIs **tend to hinder logs and useful debug/troubleshooting** information.
4. CLIs enable automated workflows.

The rule of thumb: If you **master the CLI**, GUIs hold no secrets. The opposite is not true.

Where to Access the Git CLI:

1. **Local Option (the Preferred):** Install Git client directly on your machine: **System terminal** or IDEs such as **VSCode**.
2. **Master's Data Science Hub (the Fastest for today's course):** Web-based Git CLI
3. Cloud options: **GitHub CodeSpaces** provides a VSCode-like environment (with terminal included).

Prerequisites: a **GitHub Account is required**. Create one at <https://github.com/> if you haven't already.



GETTING STARTED WITH GIT

About the Hands-on tutorial

1. We will follow the tutorial **from the GitHub repository at**
<https://github.com/orviz/git-github-tutorial-master-datascience/blob/master/README.md>
2. As student, you could copy/paste the commands but **typing the git commands is recommended** as it builds muscle memory

Gracias por su atención

DE LO MÁS PEQUEÑO A LO MÁS GRANDE

i F (A

